

Exploring Large Language Model Agents' Trajectories via Semantic Entropy

Ramtin Sirjani

Paul Moore



- **Agentic LLMs**

- LLMs as autonomous agents that interact with environments
- Reasoning and acting studied separately before ReAct
 - Chain-of-Thought: reasoning only (no grounding)
 - Act-only: actions without explicit reasoning

- **ReAct** (Yao et al., ICLR 2023)

- Interleave **thoughts** and **actions**
- Thoughts: decompose goals, track progress, handle exceptions
- Actions: interact with tools (bash, search, APIs)
- Observations: environment feedback grounds reasoning

ReAct Loop Example

Thought: I need to find where
the bug is triggered. . .

Action: bash
grep -rn "error" src/

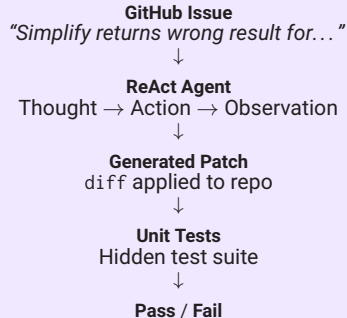
Observation:
src/core.py:42: raise Error

Thought: The issue is in
core.py line 42. . .

Action: edit src/core.py
...

- **SWE-Bench** (Jimenez et al., ICLR 2024)
 - Real GitHub issues from open-source Python repos
 - Agent reads issue, explores repo, writes a patch
 - Evaluated via held-out unit tests (pass/fail)
- **SWE-Bench Verified**
 - Human-validated 500-instance subset
 - Ensures solvable, unambiguous problems

SWE-Bench Pipeline



- **Mode Collapse in LLMs**

- Aligned models produce low-entropy distributions
- Greedy decoding amplifies homogeneity
- Same prompt → same strategy, repeatedly

- **Empirical evidence**

- **NoveltyBench** (Zhang et al., COLM 2025): frontier models produce <3 distinct responses out of 10 samples; larger models are *less* diverse

- **Consequence for agents**

- Agent gets stuck in one strategy
- More compute on the same strategy doesn't help

Greedy Agent (N runs)

Run 1: strategy A → fail

Run 2: strategy A → fail

Run 3: strategy A → fail

...

Run N: strategy A → fail

Diverse Agent (N runs)

Run 1: strategy A → fail

Run 2: strategy B → pass

Run 3: strategy C → fail

GPT-4o: 2.88 distinct out of 10 samples

Claude 3.5 Sonnet: 1.76 / 10

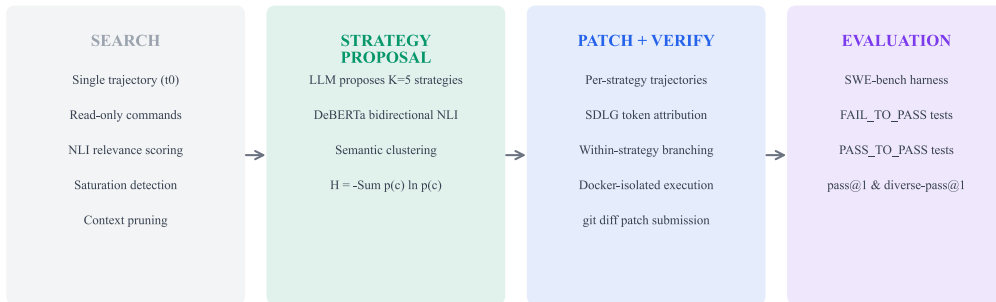
Humans: 8 distinct / 8 samples

Core Hypothesis

At each agent decision point, if the model's output distribution has high semantic entropy, branching into independent trajectories will increase the probability that at least one trajectory solves the problem.

Baseline:

- Run the vanilla Qwen3-Coder-30B-A3B, 4-bit AWQ quantization model on 10 swebench instances.
- Due to model availability and compute limitations, we selected 10 “< 15min” instances.



Allow the model to explore the environment for a minimum of 8 steps:

- Evaluate search result relevance using DeBERTa uni-directional entailment.
- Hypothesis=Issue, Premise=Summary of search result.
- Cut off search after 3 irrelevant results (relevance < 0.5).

We prompt the model to generate 5 unique strategies, then:

- Cluster the strategies using bidirectional entailment (Farquhar et al. 2024).
 - TLDR: Predict entailment using DeBERTa.
 - Mutual entailment with cluster representative $\implies$ same cluster.
- Compute semantic entropy $H = - \sum p(c) \log p(c)$ over cluster distribution.
 - $p(c) = (\text{size of cluster}) / (\text{total trajectories})$
- If $H > \tau$, fork one trajectory per unique cluster.
- Pursue each trajectory to completion.

Semantically Diverse Language Generation (SDLG) (Aichberger et al. 2025)

- After generating trajectories, feed output as self-entailment through DeBERTa.
- Compute gradient of loss toward the contradiction class ($\nabla_{z_i} L$).
- For top- K attributed positions, score substitution tokens ($S_{i,j}$).
- Use token output probability as the importance score ($l_{i,j}$).
- Regenerate from top 3 $C_{i,j}$ substitution points and recluster for diversity.

$$A_i = \|z_i \odot \nabla_{z_i} L\|_2$$

$$S_{i,j} = \frac{(z_i - z_j) \cdot \nabla_{z_i} L}{\|z_i - z_j\|_2 \|\nabla_{z_i} L\|_2}$$

$$l_{i,j} = p(v_j \mid y_{<i}, x, w)$$

$$C_{i,j} = (A_i + S_{i,j} + l_{i,j})/3$$

Preliminary Results

Resolution

Resolution results on 10 SWE-bench Verified instances (SymPy). Baseline uses a single greedy trajectory; Branching uses semantic entropy clustering with SDLG; **Union** counts instances solved by either method.

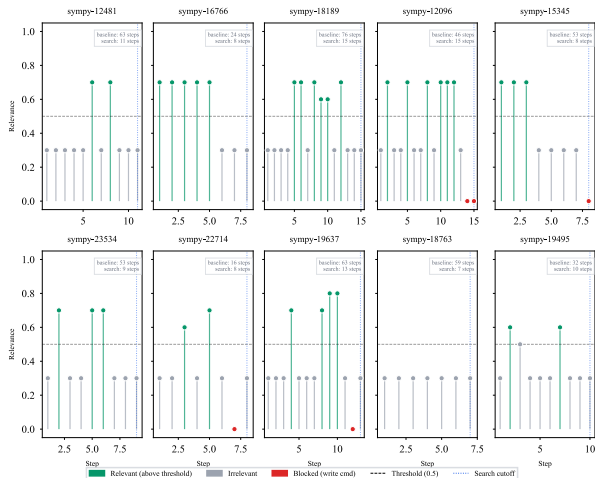
Instance	Gold (chars)	F2P /P2P	Baseline pass@1	Branching div-pass@1	Union	Solving Trajectory
sympy-12096	505	1/43	✓	✗	✓	—
sympy-12481	760	1/7	✗	✗	✗	—
sympy-15345	808	1/8	✗	✗	✗	—
sympy-16766	583	1/7	✓	✓	✓	primary, S2, S3
sympy-18189	621	1/—	✗	✗	✗	—
sympy-18763	588	1/141	✗	✗	✗	—
sympy-19495	2106	1/8	✓	✗	✓	—
sympy-19637	421	1/40	✗	✓	✓	S3_D1, S3_D2
sympy-22714	669	1/11	✗	✓	✓	primary, S1
sympy-23534	365	1/11	✗	✓	✓	primary
Total			3/10 (30%)	4/10 (40%)	6/10 (60%)	

Key: S_n = strategy branch n ; D_n = SDLG sub-branch n ; F2P/P2P = fail-to-pass / pass-to-pass. Highlighted rows are solved only by branching.

Preliminary Results

Search Relevance

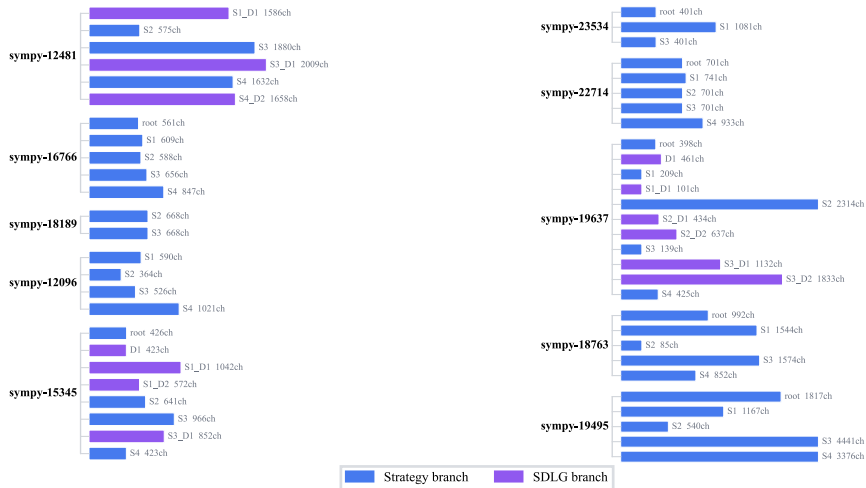
Search Phase: NLI Relevance Scoring per Step (Branching vs. Baseline Step Count)



Preliminary Results

Branching

Trajectory Branching Tree: 10 Instances, 54 Unique Patches



If time allows:

- Evaluate our method on the remaining SWE-Bench verified benchmark.
- Perform ablation study on Strategic Branching and SDLG.
- Try different models, or different levels of quantization.
- Tune hyperparameters.
- Explore non-greedy clustering.

Thank you!

Questions?

sympy-12096

sympy-12096

Gold

sympy/core/function.py

```
510 -         return Float(self._imp_(*self.args), prec)
511 +         return Float(self._imp_(*[i.evalf(prec) for i in self.args]), prec)
```

Baseline PASS

sympy/core/function.py

```
510 -         return Float(self._imp_(*self.args), prec)
511 +         # Evaluate arguments first to ensure they are numeric
511 +         evaluated_args = [arg.evalf() if hasattr(arg, 'evalf') else arg for arg in self.args]
511 +         return Float(self._imp_(*evaluated_args), prec)
```

Branching (primary) FAIL

sympy/utilities/lambdify.py

```
698 -         symfunc._imp_ = staticmethod(implementation)
699 +         symfunc._imp_ = implementation
699 +         symfunc._is_implemented_function = True
```


sympy-12481

sympy-12481

```
Gold
sympy/combinatorics/permutations.py
898 -         if has_dups(temp):
899 -             if is_cycle:
900 -                 raise ValueError('there were repeated elements; to resolve '
901 -                                 'cycles use Cycle%s.' % ''.join([str(tuple(c)) for c in args]))
902 -             else:
903 -                 raise ValueError('there were repeated elements.')
904 +         if has_dups(temp) and not is_cycle:
904 +             raise ValueError('there were repeated elements.')
```

```
Baseline FAIL
sympy/combinatorics/permutations.py
899 -             if is_cycle:
900 +             if False:
```

```
Branching (primary) FAIL
sympy/combinatorics/permutations.py
457 +     @staticmethod
457 +     def _apply_non_disjoint_cycles(cycles):
457 +         """
457 +         Apply non-disjoint cycles in left-to-right order to produce a permutation.
457 +         When cycles are non-disjoint, they should be applied sequentially
457 +         from left to right to produce the resulting permutation.
457 +         Parameters
457 +         ... +25 more lines
```

sympy-15345

sympy-15345

```
Gold
sympy/printing/mathematica.py
 35 +     "Max": [(lambda *x: True, "Max")],
 35 +     "Min": [(lambda *x: True, "Min")],
104 +     _print_MinMaxBase = _print_Function
```

```
Baseline FAIL
(no patch submitted)
```

```
Branching (primary) FAIL
sympy/printing/mathematica.py
 12 +     "Max": [(lambda *args: True, "Max")],
```

sympy-16766

sympy-16766

```
Gold
sympy/printing/pycode.py
360 +     def _print_Indexed(self, expr):
360 +         base = expr.args[0]
360 +         index = expr.args[1:]
360 +         return "{}[{}]".format(str(base), ", ".join([self._print(ind) for ind in index]))
```

```
Baseline PASS
sympy/printing/pycode.py
356 +     def _print_Indexed(self, expr):
356 +         base, *index = expr.args
356 +         return "{}[{}]".format(str(base), ", ".join([self._print(ind) for ind in index]))
```

```
Branching (primary) PASS
sympy/printing/pycode.py
361 +     def _print_Indexed(self, expr):
361 +         base, *indices = expr.args
361 +         return "{}[{}]".format(str(base), ", ".join([self._print(ind) for ind in indices]))
```

```
Branching (S2) PASS
sympy/printing/pycode.py
360 +     def _print_Indexed(self, expr):
360 +         base, *index = expr.args
360 +         return "{}[{}]".format(str(base), ", ".join([self._print(ind) for ind in index]))
```

```
Branching (S3) PASS
sympy/printing/pycode.py
356 +     def _print_Indexed(self, expr):
356 +         base, *index = expr.args
356 +         return "{}[{}]".format(str(base), ", ".join([self._print(ind) for ind in index]))
```

sympy-18189

sympy-18189

```
Gold
sympy/solvers/diophantine.py
185 -         for t in diophantine(eq, param)}
186 +         for t in diophantine(eq, param, permute=permute)}
```

Baseline FAIL
(no patch submitted)

```
Branching (primary) FAIL
sympy/solvers/diophantine.py
185 -         for t in diophantine(eq, param)}
186 +         for t in diophantine(eq, param, syms=syms, permute=permute)}
```

sympy-18763

sympy-18763

```
Gold
sympy/printing/latex.py
706 -         return r'\left. %s \right|_{\substack{ %s }}' % (latex_expr,
707 +         return r'\left. \left(%s\right) \right|_{\substack{ %s }}' % (latex_expr,
```

```
Baseline FAIL
sympy/printing/latex.py
701 -         latex_expr = self._print(expr)
702 +         latex_expr = self.parenthesize(expr, PRECEDENCE["Add"])
```

```
Branching (primary) FAIL
sympy/printing/latex.py
706 -         return r'\left. %s \right|_{\substack{ %s }}' % (latex_expr,
707 -         latex_subs)
709 +     def _print_Integral(self, expr):
710 +         # Check if we need parentheses around the expression
710 +         from sympy.printing.precedence import precedence, PRECEDENCE
710 +         if precedence(expr) < PRECEDENCE["Add"] and not isinstance(expr, (tuple, list)):
710 +             latex_expr = self.parenthesize(latex_expr, PRECEDENCE["Add"])
710 +         return r'\left. %s \right|_{\substack{ %s }}' % (latex_expr, latex_subs)
```

sympy-19495

sympy-19495

```
Gold
sympy/sets/conditionset.py
 83 -     Notes
 84 -     =====
105 -     and appears as a free symbol in the base set and the dummy symbol
106 +     appears as a free symbol in the base set and the dummy symbol
243 -         cond = self.condition.subs(old, new)
244 -         base = self.base_set.subs(old, new)
245 -         if cond is S.true:
... +10 more lines
```

```
Baseline PASS
sympy/sets/conditionset.py
246 -         return ConditionSet(new, Contains(new, base), base)
247 +         return base
```

```
Branching (primary) FAIL
sympy/sets/fancysets.py
358 -     lamda = property(lambda self: self.args[0])
359 +     lambda_ = property(lambda self: self.args[0])
397 -         val = self.lamda(*i)
398 +         val = self.lambda_(*i)
405 -         return len(self.lamda.variables) > 1
406 +         return len(self.lambda_.variables) > 1
444 -         expr = self.lamda.expr
... +7 more lines
```

sympy-19637

sympy-19637

```
Gold
sympy/core/sympify.py
516 -     hit = kern in s
517 +     hit = kern in s
517 +     else:
517 +     hit = False
```

```
Baseline FAIL
sympy/core/sympify.py
509 + kern = None
514 + hit = kern is not None and kern in s
```

```
Branching (primary) FAIL
sympy/core/sympify.py
457 +     kern = '_' # Initialize kern to avoid UnboundLocalError
```

```
Branching (S3_D1) PASS
sympy/core/sympify.py
510 -     if ' ' in s:
511 +     for i in range(2):
511 +     kern = None
511 +     # Generate kern if needed
511 +     kern = None
511 +     if ' ' in s:
512 -     kern = ' '
... +13 more lines
```

```
Branching (S3_D2) PASS
sympy/core/sympify.py
442 +     return expr
442 + # Avoid circular import
442 + from .basic import Basic
510 +     # Initialize kern to avoid UnboundLocalError
510 +     kern = None
516 -     hit = kern in s
517 +     hit = kern is not None and kern in s
... +18 more lines
```

sympy-22714

sympy-22714

```
Gold
sympy/geometry/point.py
155 -         if any(a.is_number and im(a) for a in coords):
156 +         if any(a.is_number and im(a).is_zero is False for a in coords):
```

```
Baseline FAIL
sympy/geometry/point.py
155 -         if any(a.is_number and im(a) for a in coords):
156 +         if any(a.is_number and im(a) != 0 for a in coords):
```

```
Branching (primary) PASS
sympy/geometry/point.py
155 -         if any(a.is_number and im(a) for a in coords):
156 +         if evaluate and any(a.is_number and im(a) for a in coords):
```

```
Branching (S1) PASS
sympy/geometry/point.py
155 -         if any(a.is_number and im(a) for a in coords):
156 +         if any(a.is_number and (im(a).evalf() if hasattr(im(a), "evalf") else im(a)) != 0 for a in coords):
```


sympy-23534

sympy-23534

```
Gold
sympy/core/symbol.py
794 -         result.append(symbols(name, **args))
795 +         result.append(symbols(name, cls=cls, **args))
```

```
Baseline  FAIL
(invalid patch – not a diff)
```

```
Branching (primary)  PASS
sympy/core/symbol.py
794 -         result.append(symbols(name, **args))
795 +         result.append(symbols(name, cls=cls, **args))
```